

Série de TP 1 - Python pour la modélisation mathématique

Éléments de Base

1. Demander à l'utilisateur d'entrer deux nombres réels a et b . Afficher leur somme, différence, produit, quotient, ainsi que la moyenne arithmétique $\frac{a+b}{2}$ et la moyenne géométrique $\sqrt{ab}$ (avec gestion d'erreur si $ab < 0$).
2. Écrire un programme qui demande à l'utilisateur de saisir son année de naissance, puis affiche son âge et l'année à laquelle il ou elle atteindra (ou a atteint) 100 ans.

Structures de Contrôle

1. Écrire un programme qui demande à l'utilisateur d'entrer un entier n et affiche tous les diviseurs de n , ainsi que leur somme. Vérifier si n est un nombre parfait (égal à la somme de ses diviseurs propres).
2. Écrire un programme qui utilise une boucle `for` pour afficher les entiers de 1 à 100. Pour chaque nombre, afficher "Premier" s'il est premier, "Pair" s'il est pair, "Impair" s'il est impair (priorité à "Premier").
3. Écrire un programme qui calcule une approximation de la constante d'Euler $e = \sum_{k=0}^n \frac{1}{k!}$ avec une précision donnée par l'utilisateur (arrêt lorsque le terme est inférieur à 10^{-6}). Comparer avec `math.e`.
4. (**Méthode de Héron**) Écrire un programme qui utilise une boucle `while` pour calculer une approximation de $\sqrt{a}$ (où $a > 0$ est saisi par l'utilisateur) en itérant $x_{n+1} = \frac{1}{2} \left(x_n + \frac{a}{x_n} \right)$. Afficher les itérations et s'arrêter lorsque $|x_{n+1}^2 - a| < 10^{-10}$. Comparer avec `math.sqrt(a)`.

Structures de Données

1. Créer une liste de nombres complexes (par exemple $[1 + 2j, 3 - 1j, -2 + 4j, 0 + 1j]$). Écrire un programme pour afficher leur module, leur argument, et leur conjugué. Trier la liste par module croissant.
2. Écrire un programme qui demande à l'utilisateur d'entrer des valeurs réelles dans une liste (jusqu'à ce qu'il entre une chaîne vide). Afficher la moyenne, l'écart-type (formule empirique), la médiane et les quartiles. Utiliser les fonctions natives et éventuellement `statistics` pour vérification.
3. (**Matrices avec listes de listes**) Soit une matrice carrée $A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$. Écrire des fonctions pour :
 - Calculer la trace de A .
 - Calculer la transposée de A .
 - Vérifier si A est symétrique.

4. Utiliser un dictionnaire pour représenter une fonction mathématique $f : \mathbb{R} \rightarrow \mathbb{R}$ sur un ensemble discret. Par exemple, $f(x) = x^2$ pour $x \in \{-2, -1, 0, 1, 2\}$. Écrire une fonction qui prend un dictionnaire et retourne la valeur maximale et la valeur minimale de f .
5. (**Produit scalaire**) Soient deux vecteurs représentés par des listes de même longueur. Écrire une fonction `produit_scalaire(u, v)` qui retourne leur produit scalaire. Tester avec des vecteurs aléatoires de dimension n (utiliser `random.random()`).

Fonctions

1. Écrire une fonction `binom(n, k)` qui retourne le coefficient binomial $\binom{n}{k}$ en utilisant la formule $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Ajouter une vérification que $0 \leq k \leq n$.
2. Écrire une fonction `somme_diviseurs(n)` qui retourne la somme des diviseurs propres de n . Utiliser cette fonction pour trouver tous les nombres parfaits inférieurs à 10 000.
3. (**Algorithme d'Euclide**) Écrire une fonction `pgcd(a, b)` qui calcule le PGCD de deux entiers en utilisant l'algorithme d'Euclide (itératif ou récursif). Écrire ensuite une fonction `ppcm(a, b)` utilisant la relation $\text{ppcm}(a, b) = \frac{|a \cdot b|}{\text{pgcd}(a, b)}$.
4. (**Lambda et intégration approchée**) Soit $f(x) = x^2$. Écrire une fonction `lambda` correspondante. Implémenter la méthode des rectangles (à gauche) pour approcher $\int_0^1 f(x)dx$ avec n sous-intervalles, où n est un paramètre. Comparer avec la valeur exacte $\frac{1}{3}$.
5. (**Fonction comme paramètre**) Écrire une fonction `zero_de_f(f, a, b, eps=1e-6)` qui recherche une racine d'une fonction continue f sur l'intervalle $[a, b]$ par dichotomie (on suppose $f(a) \cdot f(b) < 0$). Tester avec $f(x) = x^2 - 2$ pour trouver $\sqrt{2}$.

Facultatif

1. (**Suite de Syracuse**) Écrire une fonction `syracuse(n)` qui génère la suite de Syracuse à partir d'un entier n (si n pair, $n \leftarrow n/2$; si impair, $n \leftarrow 3n + 1$). Afficher le temps de vol et l'altitude maximale. Vérifier la conjecture pour n entre 1 et 10000.